

Spoken Digit Recognition

Practice 1 — Project Structure, Training, and Visualization

Teva Philippe - Alvaro Crespo - Amets Martiarena

1. Introduction

This first practice constitutes the first step in setting up our machine learning project. The objective is to integrate a public dataset, train a simple machine learning model, and analyze its performance.

We chose to work on digit classification. The principle is similar to **MNIST**, which allows digits to be recognized from images, but here we use audio recordings instead. We decided to use this dataset because while having the rarity of being audio files, digit recognition is a known problem and the dataset is reliable.

2. Project Structure and Dataset

In this section we will explain the project structure and make a brief explanation about the selected dataset.

Project Structure

In order to organize our project in a clear and professional manner, we used Cookiecutter Data Science to set up the project structure. Due to the small amount of extra code that is needed, we have used the structure to follow the convention, asset storing and report writing.

Dataset

We chose the **Audio MNIST** dataset. Similarly to the classic MNIST dataset, which contains images representing digits from 0 to 9, Audio MNIST contains audio recordings.

Each example is associated with a digit between 0 and 9. For each file, we also have access to metadata such as the speaker's accent, age, and gender.

We have access to 30000 recordings divided into 2 groups: 24000 samples for the training data group and 6000 samples the test data group.

It is worth noting that the data is generally balanced. However, regarding accents, we observed that the German accent was more represented in the training data, but we will not focus on this aspect here.

3. Data Exploration

The dataset has digit labels and speakers metadata information, for the classification problem only the digits labels are essential. But we will analyze some of the speakers information in order to find unbalanced data and biases that the classifier could have

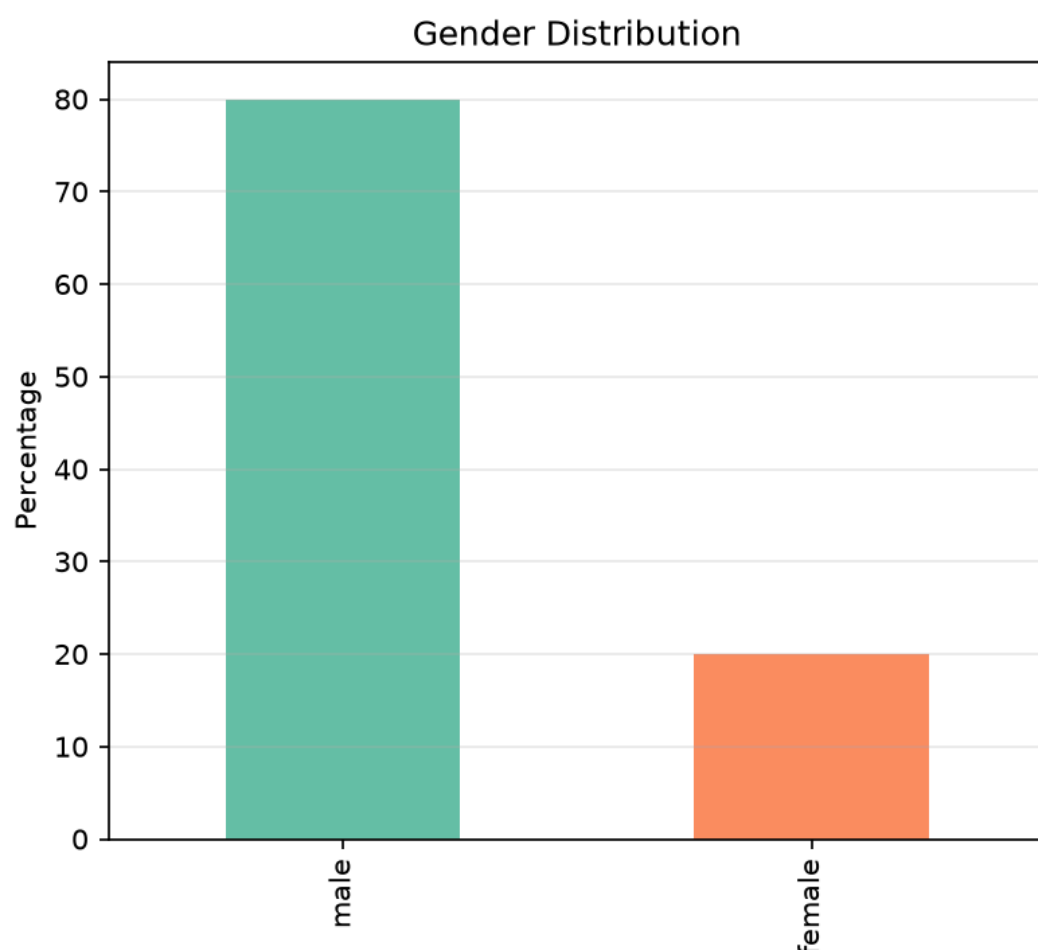


Figure 1: Bar Chart

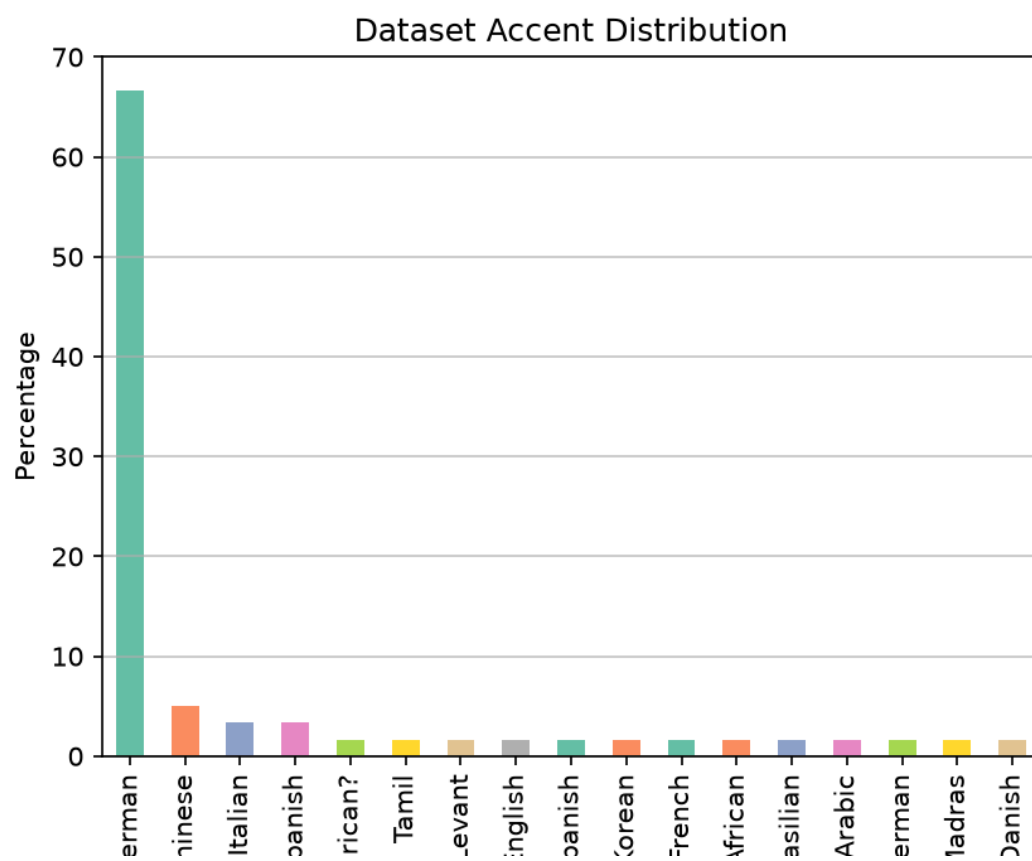


Figure 2: Boxplot

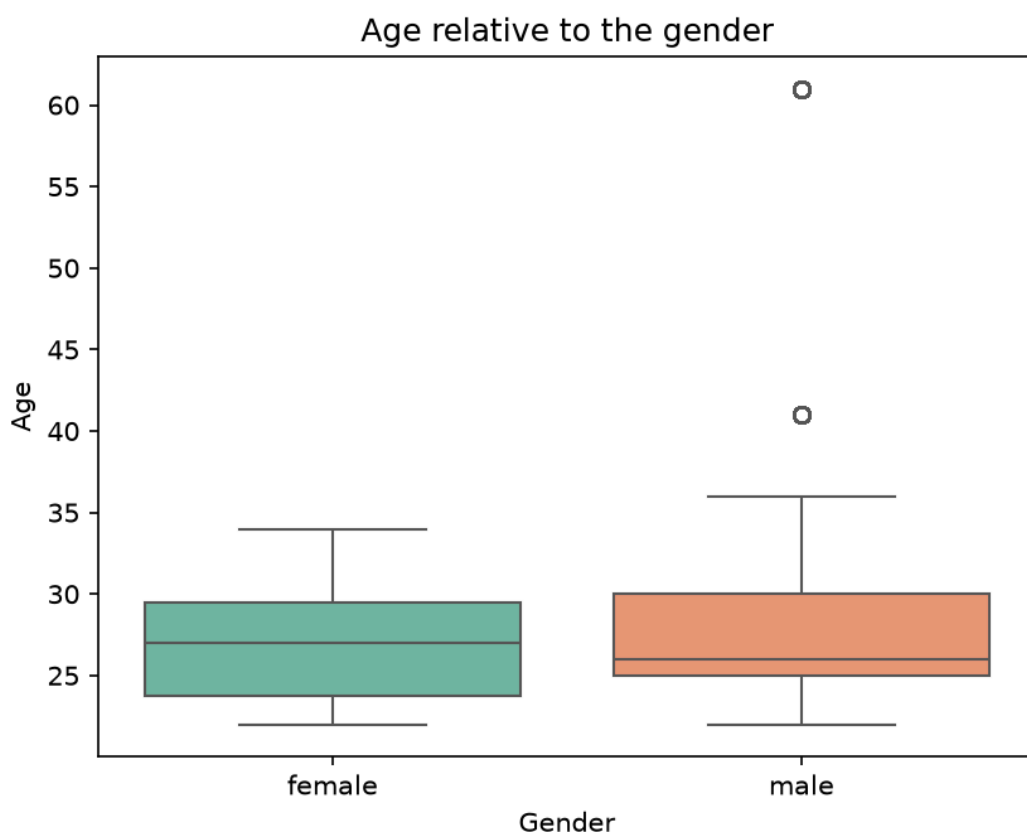


Figure 3: Boxplot

Looking to the first plot, we can see that the male speakers are far superior than the female speakers. Following with the distribution of the audios by speaker's age, most of the audios have speakers between 25 and 25 years. Finally the majority of the accents are German. As a summary we can say that this dataset has not a correct demographic representation and could be biased.

Model and Training

In this section we will introduce the model architecture that we will train as an spoken digit classifier and the training process for it.

Model Architecture

We chose to use PyTorch to build and train our classification model.

The architecture is a Multilayer Perceptron adapted to the MFCC output 26 features. The 26 features extracted from each audio recording are first passed to a fully connected layer of 64 neurons. Note that this value was chosen arbitrarily. We then reduce this to 10 neurons corresponding to the ten possible digits.

In the other hand, due to the bad results we had in the first iterations, we tried reducing the learning rate of the optimizer from $1e-3$ to $1e-4$.

It should be noted that initially, we reduced the number of neurons from 64 to 32, and then to 10. However, the results plateaued somewhat, and we hypothesize—though this should be taken with a grain of salt since we are not certain—that the model was memorizing the data. Going directly from 64 to 10 neurons seems to have resolved this issue.

After 40 epochs the training reached to a decent accuracy of 0.92.

Confusion matrix

To display the performance evolution of the model, we've saved the confusion matrices in different epochs.

In any case, in practice, we have managed to produce results that we find interesting.

We created confusion matrices for each epoch to verify the actual results produced by our model. You can see, for each training epoch (i.e., across 2,400 data points), the results produced by our model

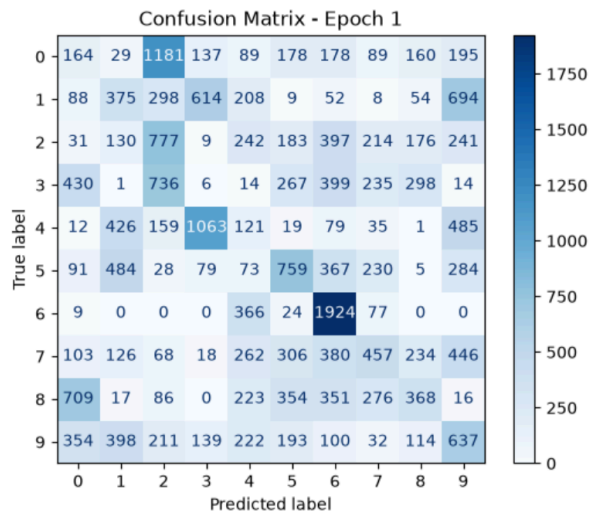


Figure 4: Confusion 1

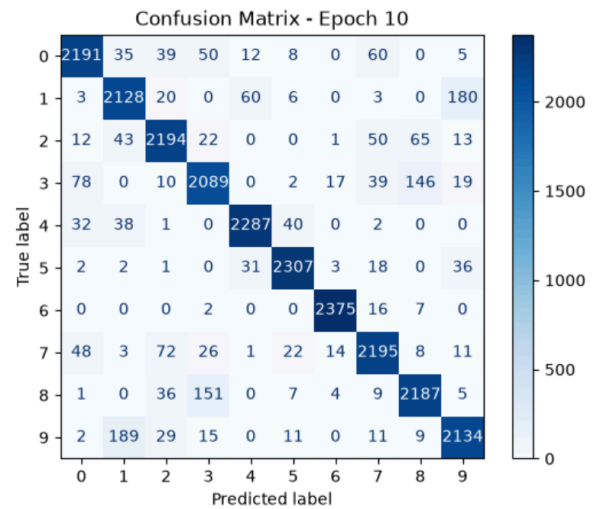


Figure 5: Confusion 10

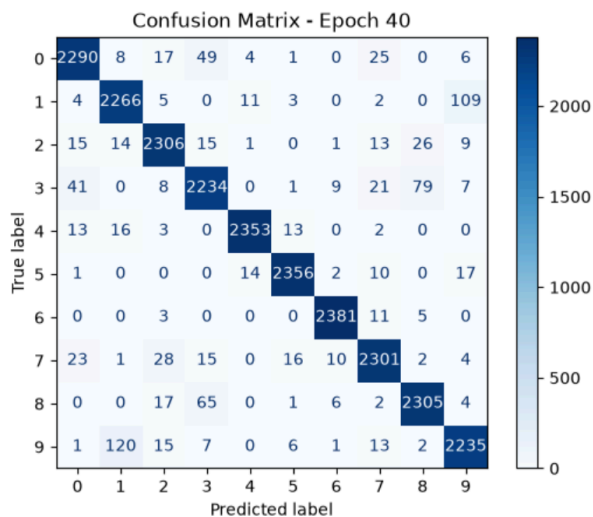


Figure 6: Confusion 40

Above, we show the changes between the first round, the 10th round, and the 40th round. The loss continues to decrease and the accuracy continues to increase even after 40 rounds, but it is especially in the first rounds that the results seem to suggest that our model is learning.

Loss / Accuracy

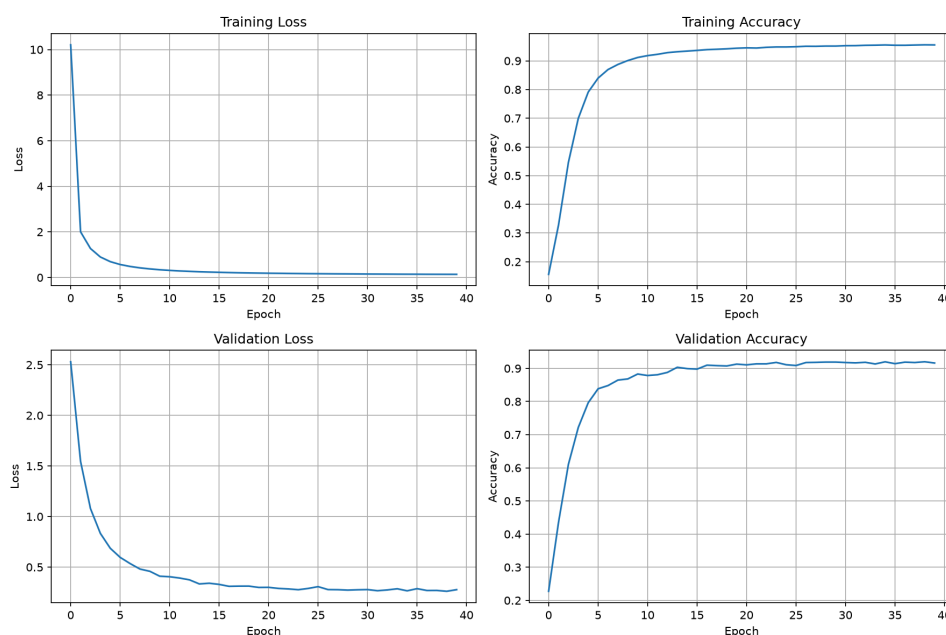


Figure 7: Training and validation loss/accuracy over 40 epochs

The curves confirm what the confusion matrices already suggested: most of the learning happens very early. The training loss drops from 7.99 at epoch 1 to 0.48 by epoch 5, while training accuracy jumps from 21.8% to 85.7% over the same span. Validation follows the same pattern, going from 39.2% to 84.0% accuracy in the first 5 epochs.

From there, both curves keep improving but at a much slower pace, and after roughly epoch 15-20 a gap opens up between training and validation performance: training loss keeps decreasing steadily down to 0.12 at epoch 40, whereas validation loss flattens out around 0.26-0.27 and even fluctuates slightly from one epoch to the next. Training accuracy reaches 96.1% by the end, while validation accuracy plateaus around 92-93% from epoch 30 onward, finishing at 92.3%. This divergence between the training and validation curves is a sign of mild overfitting, though it is not severe since validation accuracy does not degrade, it simply stops improving while the model keeps fitting the training set a bit closer.

Results suggests that training much beyond 20-25 epochs brings little extra generalization: the model reaches close to its final validation accuracy (91%) around that point already, and the remaining epochs mostly let it memorize the training data further. An early-stopping criterion on the validation loss would likely have given a comparable final result with less training time.

4. Conclusions

We built a model using Cookiecutter, analyzed a dataset of audio files using Matplotlib and Seaborn, converted the audio files into numerical data using MFCCs, and created and trained a model using PyTorch.

After changing the number of neurons and the learning rate we notice these changes improved the results and found that the model seemed to be learning but we're still not sure about the relevance of what we did.

The confusion matrices helped us understand the results provided by our model, and with the loss and accuracy graphs, we can see that our model is more accurate and more confident. It seems to us that it has “learned” as we had hoped.